

Progetto 2

Introduzione

Il progetto prevede l'implementazione di due algoritmi che hanno la funzione di individuare la presenza di almeno un ciclo all'interno di un grafo non orientato e non pesato sugli archi. Il primo di questi sfrutta l'utilizzo della struttura dati **unionFind**, mentre il secondo l'uso dell'approccio della visita in profondità, la **depth-first search (DFS)**.

Scelte implementative

In entrambi gli algoritmi i grafi di input, creati casualmente tramite un generatore, sono implementati con l'uso delle **liste di adiacenza**, conveniente sia in termini di tempo che di occupazione in memoria.

Per l'implementazione dell'algoritmo **hasCycleUF** è stata utilizzata la struttura dati **quickUnion** con bilanciamento sulle union e **path compression**, che risulta più efficiente della semplice **quickUnion** (bilanciata oppure no).

La scansione degli archi avviene con l'uso di un **iterator**, in cui un indice inizializzato a 0 viene incrementato ad ogni chiamata del metodo **__next__**. Impostato poi un attributo **max**, non appena l'indice iniziale diventa maggiore di quest'ultimo, l'iterazione viene interrotta. Inoltre, per evitare che ci siano ripetizioni (siccome un arco fra due nodi non orientato è stato considerato come due archi orientati ognuno in senso opposto all'altro), si mantiene una lista con tutti gli archi già visitati.

Il tempo di esecuzione dell'algoritmo, con al più **n** operazioni di **makeSet**, **m** operazioni di **find** ed **n-1** operazioni di **union** (dove **n** è il numero di nodi ed **m** è il numero di archi), è **$O(m + n \log(n))$** .

Per quanto concerne **hasCycleDFS**, la base dell'implementazione è quello della visita in profondità vista a lezione.

Nell'algoritmo la scelta della radice da cui far partire tale visita avviene tramite il metodo **choice** della libreria **random** di python. Successivamente viene invocata una funzione iterativa in cui vengono passati come parametri il grafo e la radice dell'albero. I nodi vengono memorizzati e selezionati in una struttura dati pila, mentre nel dizionario "**explored**" vengono inseriti i nodi incontrati durante la visita in profondità: se ad un certo punto viene preso in considerazione un nodo già presente all'interno del dizionario allora nel grafo c'è un ciclo; graficamente un ciclo può essere individuato con un arco all'indietro nell'albero.

Il tempo di esecuzione dell'algoritmo, sia se il grafo è aciclico sia nel caso in cui è presente un ciclo, è **$O(n)$** , dove n è il numero di nodi. Infatti nel primo caso, essendo il grafo connesso, il numero di archi in esso presenti è al più **$n-1$** , mentre nel secondo caso il numero di archi deve essere almeno **n** .

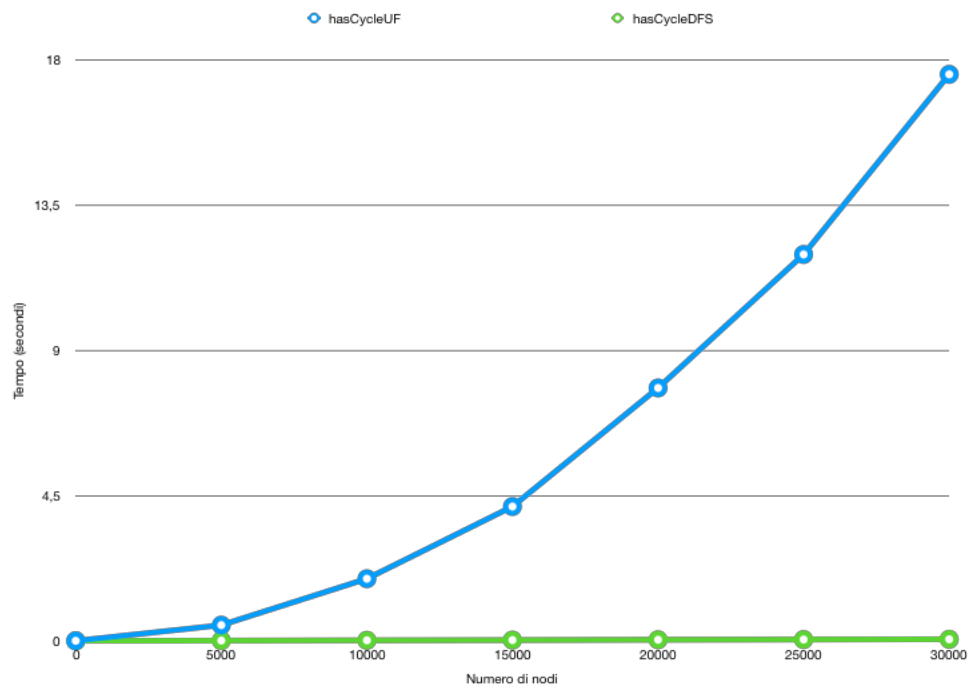
Entrambe le funzioni hanno prefisso un **decorator**, implementato in modo simile a quanto già visto, il quale, nel momento di invocazione delle stesse, scrive il nome dell'algoritmo in questione, numero di archi e tempo di esecuzione.

La funzione per generare grafi, con o senza cicli, è stata implementata in modo tale da accettare tre parametri (numero di nodi, ciclicità -sotto forma di variabile booleana-, ed eventualmente presenza di più di un ciclo o meno -impostato di default a False). In particolare, affinché vengano rispettate tali condizioni, si è imposto, tramite un if che ne ferma l'esecuzione tramite **break**, che l'arco abbia esattamente $n-1$ archi nel caso in cui lo si voglia aciclico (essendo tra l'altro connesso e non orientato), tra n e $n(n-1)$ (numero massimo di archi per nodo) altrimenti.

Risultati sperimentali:

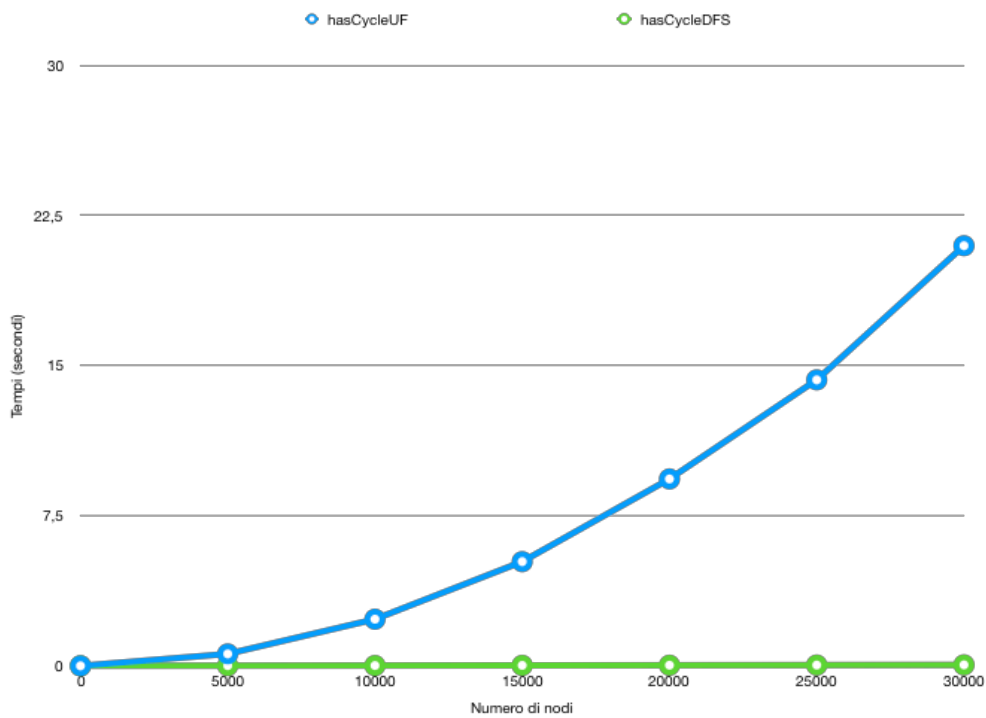
Grafo ciclico

	hasCycleUF	hasCycleDFS
0	0	0
5000	0,48534584045410156	0,007554054260253906
10000	1,9235899448394775	0,015341043472290039
15000	4,160090208053589	0,022716760635375977
20000	7,833909034729004	0,03284001350402832
25000	11,970459222793579	0,03976297378540039
30000	17,55047583580017	0,04793286323547363



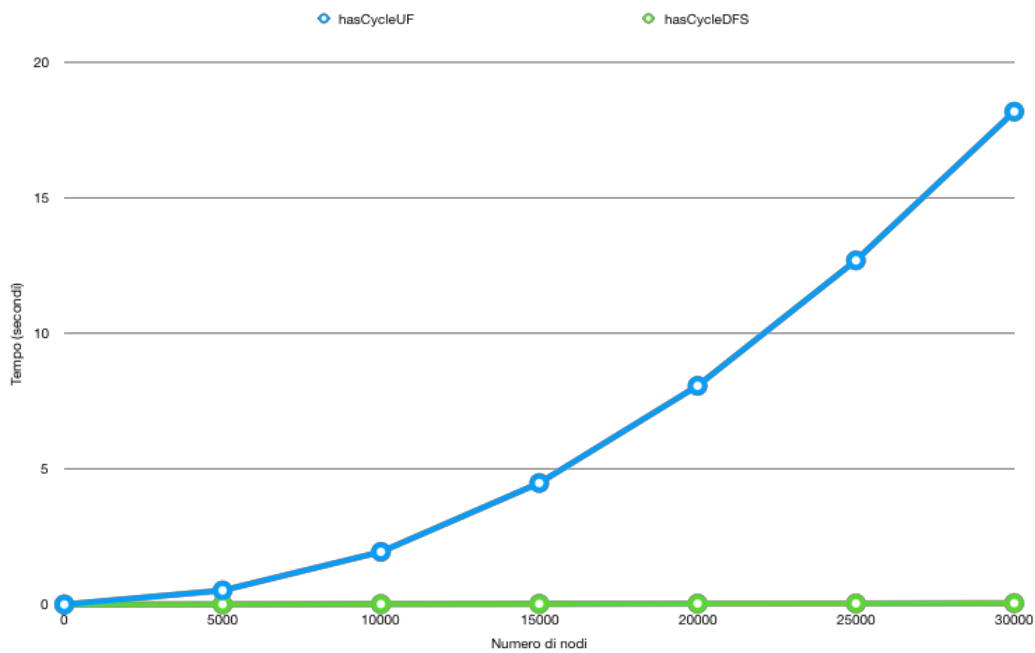
Grafo aciclico:

	hasCycleUF	hasCycleDFS
0	0	0
5000	0,5967788696289062	0,008668184280395508
10000	2,3312623500823975	0,01583719253540039
15000	5,206685304641724	0,02387404441833496
20000	9,337382078170776	0,0313570499420166
25000	14,288427114486694	0,03932690620422363
30000	20,99057674407959	0,048268795013427734



Grafo ciclico (con più di un ciclo):

	hasCycleUF	hasCycleDFS
0	0	0
5000	0,5183529853820801	0,008207082748413086
10000	1,9398510456085205	0,01593613624572754
15000	4,476700067520142	0,023726940155029297
20000	8,061227083206177	0,032533884048461914
25000	12,683591842651367	0,03776717185974121
30000	18,168895959854126	0,048454999923706055



Commento dei risultati ottenuti:

Come di evince dai risultati ottenuti, **hasCycleDFS** risulta di gran lunga più efficiente di **hasCycleUF** in tutte le situazioni. Ciò è dovuto ai migliori tempi di esecuzione che caratterizzano le singole operazioni delle strutture dati e degli approcci implementati negli algoritmi stessi: infatti se da una parte **hasCycleDFS** opera come una visita **DFS**, da cui il nome, con tempi di esecuzione lineari, essendo al più n gli archi da analizzare come osservato in precedenza, **hasCycleUF** deve inizialmente inizializzare la struttura dati union-find (attraverso il metodo **makeSet**) per poi scandire tutti gli archi non visitati nel grafo ed effettuare eventualmente operazioni di **find** o di **union**, dunque con tempi di esecuzione asintotici di $O(m + n \log(n))$.

Inoltre i tempi di esecuzione di **hasCycleUF** in generale sono risultati leggermente migliori nel caso dei grafi ciclici, dal momento che, come per **hasCycleDFS**, la sua esecuzione viene subito arrestata già nell'individuazione di un solo ciclo.

Non si notano invece sostanziali differenze per quanto riguarda **hasCycleDFS**.

Infine i tempi di esecuzione degli algoritmi, nei casi di un grafo ciclico con uno o più cicli, non subiscono sostanziali variazioni (per i motivi soprammenzionati).